

PRASA CONNECT

API and Database Hosting Guide

Detailed instructions for completing the missing Part 2 hosted REST API and database requirements.

Recommended stack: Supabase PostgreSQL + Render Node/Express API + Android Retrofit.

Student project	OPSC6312 PRASA Connect Android App
Purpose	Show real hosted data, registration/login support, bookings, incident reports and API/database evidence for the demo video.

1. What Must Be Fixed for Part 2

Your marker feedback said the app could not register, login, show real data interaction, use an API, make bookings, or logout. The Android app now has a local functional prototype, but for stronger Part 2 compliance you should host the data layer online and connect the app to it.

Requirement	What to show in the demo	Where it will be implemented
Register/Login	Create a user and log in successfully.	Android app + API auth endpoints
Logout	User returns to login screen.	Android app session state
API	Open hosted URL and show JSON responses.	Render Node/Express service
Database	Show rows created in hosted database.	Supabase PostgreSQL tables
Bookings	Book a train and show ticket saved online.	POST /bookings and tickets table
Incident reports	Submit a report and show it saved online.	POST /incidents and incidents table
Settings	Change language/notifications/offline sync.	PATCH /users/:id/settings
README/video	Explain setup and show all evidence.	GitHub README and video link

2. Recommended Hosting Architecture

Use this path because it is simple to explain and easy to demonstrate: Supabase hosts the PostgreSQL database, Render hosts the Node/Express REST API, and the Android app calls the API using Retrofit. This gives you visible evidence for both the API and the database.

Layer	Tool	Why
Android app	Kotlin + Jetpack Compose + Retrofit	Calls online API and displays real responses.
REST API	Node.js + Express hosted on Render	Easy endpoints for auth, schedules, bookings and incidents.
Database	Supabase PostgreSQL	Hosted database with tables visible in a browser dashboard.
Security	bcrypt + environment variables	Passwords are hashed and secrets are not committed to GitHub.

3. Create the Supabase Database

3.1 Create the project

- ? Go to <https://supabase.com> and create a project named prasa-connect.
- ? Save the database password somewhere safe. You will need it for the connection string.
- ? Open Project Settings -> Database -> Connect and copy the Session pooler connection string. Supabase documentation says the connection string is available from the Dashboard by clicking Connect and choosing the pooler/session option.
- ? Do not put the database password directly in code. It will be stored as an environment variable on Render.

3.2 Create tables

In Supabase, open SQL Editor and run this schema. It gives you the exact tables needed for the PRASA app demo.

```
create table if not exists app_users (  
  id uuid primary key default gen_random_uuid(),  
  full_name text not null,  
  email text unique not null,  
  mobile text not null,  
  password_hash text not null,  
  language text default 'English',
```

```

        notifications_enabled boolean default true,
        offline_sync_enabled boolean default true,
        created_at timestampz default now()
    );

create table if not exists schedules (
    id uuid primary key default gen_random_uuid(),
    train text not null,
    origin text not null,
    destination text not null,
    depart_time text not null,
    arrive_time text not null,
    platform text not null,
    status text default 'On time'
);

create table if not exists tickets (
    id uuid primary key default gen_random_uuid(),
    user_id uuid references app_users(id) on delete cascade,
    schedule_id uuid references schedules(id),
    route text not null,
    train text not null,
    fare numeric(10,2) not null,
    qr_code text not null,
    status text default 'Active',
    created_at timestampz default now()
);

create table if not exists incidents (
    id uuid primary key default gen_random_uuid(),
    user_id uuid references app_users(id) on delete cascade,
    type text not null,
    location text not null,
    description text not null,
    status text default 'Submitted',
    created_at timestampz default now()
);

```

3.3 Insert sample PRASA data

```

insert into schedules (train, origin, destination, depart_time, arrive_time, platform, status) values
('T0507', 'Cape Town', 'Bellville', '08:45', '09:15', '2', 'On time'),
('T0509', 'Cape Town', 'Bellville', '09:15', '09:45', '1', 'On time'),
('T0612', 'Johannesburg', 'Pretoria', '10:00', '10:58', '4', 'Delayed 5 min'),
('T0718', 'Pretoria', 'Ekurhuleni', '11:30', '12:15', '3', 'On time');

```

4. Create the Node/Express API

4.1 Create a new backend folder

Create a separate folder, for example prasa-connect-api. It can be a new GitHub repository or a backend folder inside the same repository.

```

mkdir prasa-connect-api
cd prasa-connect-api
npm init -y
npm install express cors dotenv pg bcryptjs uuid
npm install --save-dev nodemon

```

4.2 package.json scripts

```

{
  "name": "prasa-connect-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "latest",
    "cors": "latest",
    "dotenv": "latest",
    "express": "latest",
    "pg": "latest",
    "uuid": "latest"
  }
}

```

```

    },
    "devDependencies": {
      "nodemon": "latest"
    }
  }
}

```

4.3 .env file for local testing

Create .env locally. Do not commit it to GitHub. Add .env to .gitignore.

```

DATABASE_URL=postgresql://postgres.[PROJECT_REF]:[PASSWORD]@[POOLER_HOST]:6543/postgres
PORT=3000

```

4.4 server.js

This API supports register, login, schedules, bookings, incidents and settings. It is deliberately simple for the student prototype and demo video.

```

const express = require('express');
const cors = require('cors');
const bcrypt = require('bcryptjs');
const { Pool } = require('pg');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false }
});

app.get('/', (req, res) => {
  res.json({ ok: true, service: 'PRASA Connect API', status: 'online' });
});

app.post('/api/auth/register', async (req, res) => {
  try {
    const { fullName, email, mobile, password } = req.body;
    if (!fullName || !email || !mobile || !password) {
      return res.status(400).json({ ok: false, message: 'All fields are required' });
    }
    if (password.length < 6) {
      return res.status(400).json({ ok: false, message: 'Password must be at least 6 characters' });
    }
    const passwordHash = await bcrypt.hash(password, 10);
    const result = await pool.query(
      `insert into app_users (full_name, email, mobile, password_hash)
      values ($1, $2, $3, $4)
      returning id, full_name, email, mobile, language, notifications_enabled, offline_sync_enabled`,
      [fullName, email.toLowerCase(), mobile, passwordHash]
    );
    res.status(201).json({ ok: true, user: result.rows[0] });
  } catch (err) {
    if (String(err.message).includes('duplicate')) {
      return res.status(409).json({ ok: false, message: 'Email already registered' });
    }
    res.status(500).json({ ok: false, message: err.message });
  }
});

app.post('/api/auth/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const result = await pool.query('select * from app_users where email = $1', [email.toLowerCase()]);
    const user = result.rows[0];
    if (!user || !(await bcrypt.compare(password, user.password_hash))) {
      return res.status(401).json({ ok: false, message: 'Invalid email or password' });
    }
    delete user.password_hash;
    res.json({ ok: true, user });
  } catch (err) {
    res.status(500).json({ ok: false, message: err.message });
  }
});

app.get('/api/schedules', async (req, res) => {

```

```

const { from = '', to = '' } = req.query;
const result = await pool.query(
  `select * from schedules
   where lower(origin) like lower($1) and lower(destination) like lower($2)
   order by depart_time`,
  [`%${from}%`, `%${to}%`]
);
res.json({ ok: true, schedules: result.rows });
});

app.post('/api/bookings', async (req, res) => {
  try {
    const { userId, scheduleId } = req.body;
    const scheduleResult = await pool.query('select * from schedules where id = $1', [scheduleId]);
    const schedule = scheduleResult.rows[0];
    if (!schedule) return res.status(404).json({ ok: false, message: 'Schedule not found' });

    const qrCode = `QR-${schedule.train}-${Date.now()}`;
    const result = await pool.query(
      `insert into tickets (user_id, schedule_id, route, train, fare, qr_code)
       values ($1, $2, $3, $4, $5, $6) returning *`,
      [userId, scheduleId, `${schedule.origin} -> ${schedule.destination}`, schedule.train, 28.00, qrCode]
    );
    res.status(201).json({ ok: true, ticket: result.rows[0] });
  } catch (err) {
    res.status(500).json({ ok: false, message: err.message });
  }
});

app.get('/api/users/:userId/tickets', async (req, res) => {
  const result = await pool.query('select * from tickets where user_id = $1 order by created_at desc', [req.params.userId]);
  res.json({ ok: true, tickets: result.rows });
});

app.post('/api/incidents', async (req, res) => {
  try {
    const { userId, type, location, description } = req.body;
    if (!userId || !type || !location || !description) {
      return res.status(400).json({ ok: false, message: 'Missing incident details' });
    }
    const result = await pool.query(
      `insert into incidents (user_id, type, location, description)
       values ($1, $2, $3, $4) returning *`,
      [userId, type, location, description]
    );
    res.status(201).json({ ok: true, incident: result.rows[0] });
  } catch (err) {
    res.status(500).json({ ok: false, message: err.message });
  }
});

app.patch('/api/users/:userId/settings', async (req, res) => {
  const { language, notificationsEnabled, offlineSyncEnabled } = req.body;
  const result = await pool.query(
    `update app_users set language = $1, notifications_enabled = $2, offline_sync_enabled = $3
     where id = $4
     returning id, full_name, email, mobile, language, notifications_enabled, offline_sync_enabled`,
    [language, notificationsEnabled, offlineSyncEnabled, req.params.userId]
  );
  res.json({ ok: true, user: result.rows[0] });
});

const port = process.env.PORT || 3000;
app.listen(port, () => console.log(`PRASA Connect API running on port ${port}`));

```

5. Test the API Locally

Start the server locally and open <http://localhost:3000> in your browser. You should see JSON saying the service is online.

```
npm run dev
```

Use Postman, Thunder Client or curl to test each endpoint.

```
# Register
curl -X POST http://localhost:3000/api/auth/register ^
-H "Content-Type: application/json" ^
```

```

-d "{\"fullName\":\"Nonhlanhla Chirwa\",\"email\":\"nonhlanhla@test.com\",\"mobile\":\"0712345678\",\"password\":\"password123\"}"

# Login
curl -X POST http://localhost:3000/api/auth/login ^
-H "Content-Type: application/json" ^
-d "{\"email\":\"nonhlanhla@test.com\",\"password\":\"password123\"}"

# Schedules
curl "http://localhost:3000/api/schedules?from=Cape%20Town&to=Bellville"

```

6. Deploy the API on Render

Render official docs describe deploying a Node/Express app as a Web Service, and state that the service becomes live at its onrender.com URL after deploy finishes. Render Web Services also support environment variables/secrets from the dashboard.

- ? Push the backend code to GitHub.
- ? Go to <https://render.com> and create a New Web Service.
- ? Connect your GitHub repository.
- ? Runtime: Node. Build command: npm install. Start command: npm start.
- ? Add environment variable DATABASE_URL with the Supabase connection string.
- ? Add NODE_ENV=production if desired.
- ? Deploy. Wait until logs show the API running.
- ? Open <https://your-service-name.onrender.com> and confirm JSON response.

Render settings checklist

Setting	Value
Service type	Web Service
Runtime	Node
Build command	npm install
Start command	npm start
Environment variable	DATABASE_URL = Supabase connection string
Health check	/ or /api/schedules

7. Connect Android to the Hosted API

7.1 Add dependencies

In app/build.gradle.kts add Retrofit and Gson converter. Sync Gradle afterwards.

```

dependencies {
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
}

```

7.2 Add Internet permission

In AndroidManifest.xml add this above the application tag:

```

<uses-permission android:name="android.permission.INTERNET" />

```

7.3 Create API models

```

data class RegisterRequest(val fullName: String, val email: String, val mobile: String, val password: String)
data class LoginRequest(val email: String, val password: String)
data class BookingRequest(val userId: String, val scheduleId: String)

```

```

data class IncidentRequest(val userId: String, val type: String, val location: String, val description: String)
data class SettingsRequest(val language: String, val notificationsEnabled: Boolean, val offlineSyncEnabled: Boolean)

data class ApiUser(val id: String, val full_name: String, val email: String, val mobile: String, val language: String)
data class ApiSchedule(val id: String, val train: String, val origin: String, val destination: String, val departure: String)
data class ApiTicket(val id: String, val route: String, val train: String, val fare: Double, val qr_code: String, val user_id: String)

```

7.4 Create Retrofit service

```

interface PrasaApiService {
    @POST("api/auth/register")
    suspend fun register(@Body body: RegisterRequest): RegisterResponse

    @POST("api/auth/login")
    suspend fun login(@Body body: LoginRequest): LoginResponse

    @GET("api/schedules")
    suspend fun schedules(@Query("from") from: String, @Query("to") to: String): ScheduleResponse

    @POST("api/bookings")
    suspend fun book(@Body body: BookingRequest): TicketResponse

    @GET("api/users/{userId}/tickets")
    suspend fun tickets(@Path("userId") userId: String): TicketListResponse

    @POST("api/incidents")
    suspend fun incident(@Body body: IncidentRequest): IncidentResponse

    @PATCH("api/users/{userId}/settings")
    suspend fun settings(@Path("userId") userId: String, @Body body: SettingsRequest): LoginResponse
}

```

7.5 Retrofit client

```

object ApiClient {
    private const val BASE_URL = "https://your-service-name.onrender.com/"

    val service: PrasaApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(PrasaApiService::class.java)
    }
}

```

7.6 Replace local repository gradually

- ? First replace login/register calls with Retrofit calls.
- ? Then replace timetable search with GET /api/schedules.
- ? Then replace booking creation with POST /api/bookings.
- ? Then replace incident report submission with POST /api/incidents.
- ? Keep the UI screens you already have. Only swap the data layer from local PrasaApi to Retrofit.

8. What to Record in the Demo Video

Your video must prove the API and database are real. Use this sequence.

Step	What to show
1	Open hosted Render URL in browser and show API online JSON.
2	Open Supabase table editor and show empty/users/tickets/incidents tables.
3	Run Android app and register a new user.
4	Refresh Supabase app_users table and show the new row.
5	Log out and log in with that user.

Step	What to show
6	Search timetable and book a train.
7	Refresh tickets table and show the ticket row.
8	Submit an incident report.
9	Refresh incidents table and show the incident row.
10	Open app settings, change language/sync, and show saved result.
11	Show README and GitHub repository.

9. GitHub Actions for Build Evidence

Add this file at `.github/workflows/android-build.yml`. It proves the app builds outside your computer.

```
name: Android Build

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17
      - uses: gradle/actions/setup-gradle@v4
      - name: Build debug APK
        run: ./gradlew assembleDebug --no-daemon
      - name: Run unit tests
        run: ./gradlew test --no-daemon
```

10. Submission Checklist

- ? Hosted Render API URL works in a browser.
- ? Supabase tables contain user, schedule, ticket and incident data.
- ? Android app can register, login, logout, book, show tickets and submit incidents.
- ? README includes API URL, database choice, demo login and video link.
- ? GitHub Actions build passes.
- ? Demo video shows API URL, database tables, app workflow and voice-over explanation.
- ? Do not commit `.env`, database password, API secrets or `google-services.json` if it contains private keys.

11. References

Render. 2026. Deploy a Node Express App on Render. <https://render.com/docs/deploy-node-express-app>

Render. 2026. Web Services documentation. <https://render.com/docs/web-services>

Render. 2026. Environment variables documentation. <https://render.com/docs/environment-variables>

Supabase. 2026. Connect to your PostgreSQL database.
<https://supabase.com/docs/guides/database/connecting-to-postgres>

Supabase. 2026. Postgres.js connection guide. <https://supabase.com/docs/guides/database/postgres-js>

Firebase. 2026. Authenticate with Firebase using password-based accounts on Android.
<https://firebase.google.com/docs/auth/android/password-auth>